
White-Box State-Transition Governance for AI Agents

Candidate-First, Evidence-Bound Control of External Side Effects

Technical Research Note v0.2.2 — Public Release Candidate

Author: 万根

Affiliation: Independent Researcher

Correspondence: 593315006@qq.com

Date: October 2026

Abstract

AI agents are increasingly moving from text generation toward systems that can invoke tools, modify digital artifacts, interact with services, and initiate changes in external environments. Once an agent can produce side effects, governance cannot be reduced to the question of whether a tool call is syntactically valid or whether a subject has permission to invoke an operation. A second question becomes important: **given the current state, available evidence, policy version, and proposed change, is this particular state transition eligible to proceed?**

This technical note introduces **White-Box State-Transition Governance**, a candidate-first and evidence-bound reference model for agent-induced external state changes. The governed unit is not a generic tool invocation. It is the bound lineage formed by a Candidate Transition, its pre-state snapshot, supporting evidence, policy snapshot, governance decision, controlled execution boundary, observed outcome, and replayable lineage record. The term *white-box* is used narrowly: it does not claim access to an LLM's hidden reasoning or chain-of-thought. It refers only to the inspectability of this external governance lineage.

A minimal deterministic local reference implementation is used to evaluate the model. It does not import the protected engineering system that motivated this work, does not connect to networks, MCP, providers, shells, databases, filesystems, or real external tools, and uses a pure in-memory controlled executor with

`external_side_effects = false`. Seven cases were declared in advance, covering admission, denial, insufficient evidence, escalation, failed execution after admission, replay without re-execution, and attempted direct-tool access. The reference implementation matched all predeclared fixed cases: expected decision agreement was 7/7, expected observation agreement was 7/7, and replay verification was 1/1. The implementation also passed 14 tests under Python 3.9 and 14 tests under Python 3.13.

These results are intentionally narrow. **The result demonstrates conformance of this deterministic reference model to its declared protocol.** It does not establish production safety, real-world generalization, kernel-level isolation, independent security certification, or governance of real external side effects. A larger protected engineering environment motivated parts of the problem formulation, but that system is not required to reproduce the claims of this note and is not treated as experimental evidence here.

Evidence-Status Vocabulary

This note uses three evidence levels throughout:

`DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION`

Claims directly supported by the deterministic reference implementation and its declared tests.

`INTERNALLY_IMPLEMENTED_NOT_PUBLICLY_REPRODUCED`

Protected engineering capabilities described only at a high capability level. They are not evidence for the experiments reported in this note.

`RESEARCH_DIRECTION`

Future work not claimed as implemented, reproduced, or experimentally established in this release.

1. Introduction

1.1 From Tool Use to External State Change

Traditional language-model interaction can be simplified as:

User



LLM
↓
Answer

Agentic systems introduce another boundary:

User
↓
Agent
↓
Tool
↓
External Environment

Once a tool can change an external object, a successful API call, shell command, database mutation, message send, configuration update, or workflow transition may alter a state outside the model itself.

Existing computer-security mechanisms already address important aspects of this problem. Access-control systems determine whether subjects may perform operations on protected objects. Runtime monitors enforce classes of security policies. Policy engines separate policy decisions from application-specific enforcement. Sandboxes constrain processes, files, network access, credentials, and other resources. These mechanisms remain necessary. [1–7]

This note focuses on a narrower semantic gap:

Permission to attempt an operation is not identical to
qualification for a particular state change, and
qualification is not identical to observed success.

Consider three statements:

The agent may call an operation.
The candidate state change is eligible to proceed.
The external state change was actually applied.

They describe different events.

The proposed model therefore separates:

Proposal
≠ Governance Admission
≠ Execution
≠ Observed Success

and, more specifically:

Candidate $\neq$ Commit

ALLOW $\neq$ APPLIED

1.2 Contributions

This note makes two public technical contributions.

1. **A candidate-first, evidence-bound semantic model for state-transition governance.**

The model treats a candidate transition, pre-state snapshot, evidence binding, policy snapshot, decision, observation, and replayable lineage as one governed semantic unit.

2. **A deterministic reference demonstration and fixed-case property-verification protocol.**

The implementation exercises the declared semantics without networking, real tools, or external side effects.

The experimental claims of this paper are limited to these two contributions.

A larger protected engineering environment involving runtime evidence, derived state, candidate handling, observation, and continuity-oriented review motivated the research problem. That environment is neither reproduced nor benchmarked in this note and is not required to verify the public claims.

1.3 What This Paper Does Not Claim

The work does not claim access to hidden LLM reasoning, chain-of-thought, or internal model cognition. It does not claim to replace access control, policy engines, sandboxes, capability security, or kernel-level execution boundaries.

It also does not claim that the complete protected engineering system has been publicly reproduced, that the reference implementation is production-ready, or that the method-evolution chain:

Episode → Pattern → Method → Operator

is part of this paper.

2. Problem Statement and Threat Boundary

2.1 The Governed Unit

The central abstraction of this note is:

```
Governed Unit
=
Candidate Transition
+ Pre-State Snapshot
+ Evidence Binding
+ Policy Snapshot
+ Governance Decision
+ Observation
+ Replayable Lineage
```

A tool invocation is an implementation event.

A governed transition is the semantic unit under evaluation.

This distinction matters because one tool call may have different governance meanings under different state versions, evidence, subjects, policy snapshots, or expected post-states.

For example, an operation named `update_order` is not sufficiently descriptive by itself. The governance meaning depends on questions such as:

```
Which order?
What is its current version?
What state is it currently in?
What post-state is proposed?
Which evidence supports the change?
Which policy version applies?
Who is requesting the transition?
What was actually observed after execution?
```

2.2 Threat Boundary

The reference implementation evaluates semantic governance properties, not operating-system security.

The implementation is:

```
local
deterministic
in-memory
side-effect-free
```

It does not connect to:

```
Network
MCP
Model providers
Shell
Database
Filesystem executor
External APIs
Physical devices
Production credentials
```

The controlled executor is explicitly configured such that:

```
external_side_effects = false
```

Accordingly, the experiments do not establish safety for real file writes, API calls, database changes, financial operations, physical actuators, or other production side effects.

2.3 Agent-Facing Boundary

The Agent Gateway accepts only candidate-transition submissions.

It does not intentionally expose:

```
tool handles
executor handles
shell handles
database clients
filesystem writers
network clients
```

A direct-tool request made through the public reference API is rejected and produces an audit record.

This demonstrates an **API-level architectural property** of the reference implementation. It does not establish protection against arbitrary malicious Python code executing with the same process authority.

2.4 Why This Boundary Matters

A semantic governance layer that can simply be bypassed is only advisory.

Runtime-enforcement research has long distinguished the expression of a policy from the mechanism that mediates execution. Schneider's execution-monitoring model formalized classes of enforceable security policies, while modern systems such as OpenShell place trusted enforcement outside the untrusted agent workload. [3,11] OpenShell's documented architecture separates a trusted supervisor from the sandboxed agent workload and mediates filesystem, process, network, and credential behavior.

The present implementation does not reproduce such a hard execution boundary. Its scope is the semantic model above that boundary.

3. White-Box Governance Lineage Model

3.1 Reference Governance Lineage

Figure 1. Reference Governance Lineage Demonstrated in the Reference Implementation

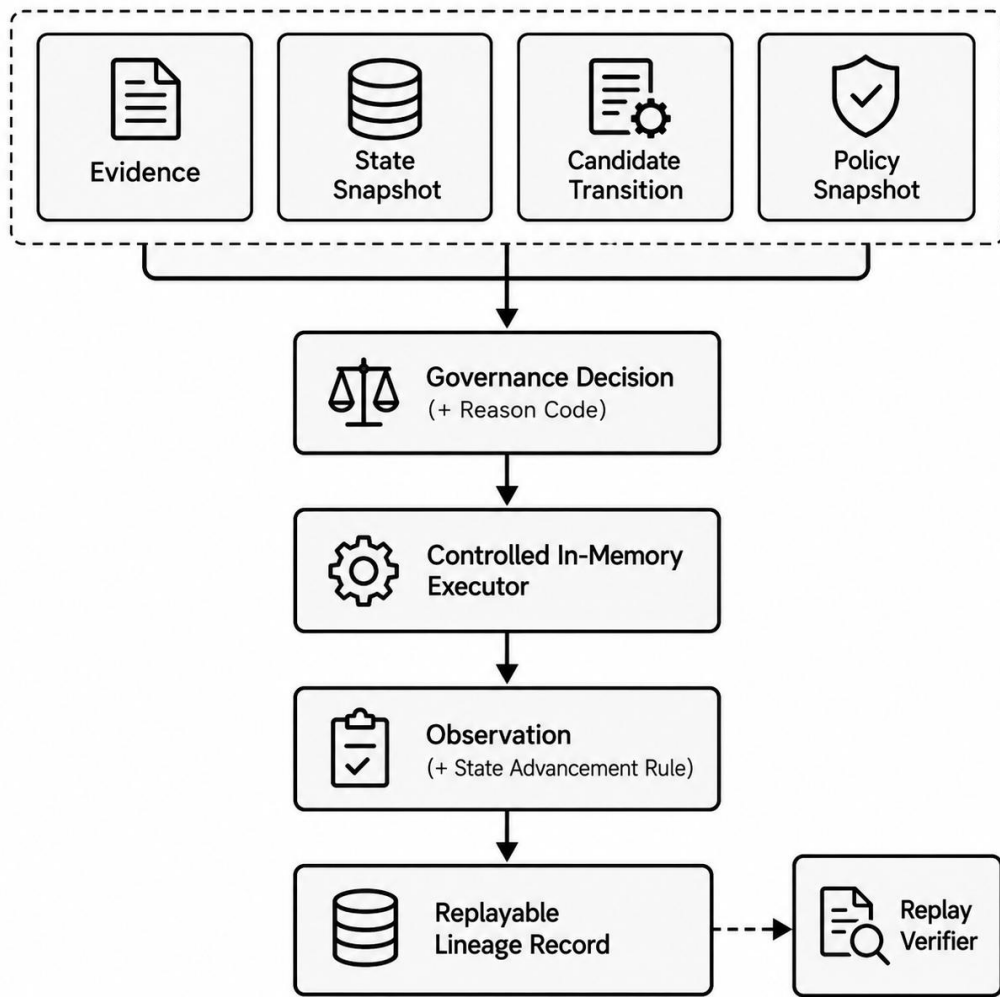


Figure 1 caption. This figure represents the reference model demonstrated by the current implementation. The executor is simulated and no real external side effect occurs.

3.2 Evidence

Evidence is information explicitly bound to the transition being governed.

The implementation fingerprints the evidence representation used for the decision. An idempotency replay is accepted only when the candidate, evidence, policy snapshot, and original state snapshot all match the first submission. A changed bound input is rejected rather than silently returning the earlier ALLOW record.

The reference implementation tests rejection when evidence is bound to the wrong object or version.

Evidence binding does **not** prove that the original evidence source is truthful. Trustworthiness of evidence sources is outside this paper's demonstrated claims.

3.3 State Snapshot

The State Snapshot represents the versioned pre-state against which the candidate is evaluated.

The state snapshot is fingerprinted.

A change in state identity or version changes the governance context.

The paper therefore distinguishes:

current state at decision time

from:

whatever state exists later

3.4 Candidate Transition

The agent does not submit an instruction semantically equivalent to:

```
execute_this_now()
```

Instead, it submits a candidate describing a proposed change.

A conceptual representation is:

```
candidate_transition:
  subject: agent_007
  object_id: object_123
  pre_state_version: v4
  operation: UPDATE
  expected_post_state: v5
  evidence_refs:
    - evidence_001
  risk_class: MEDIUM
  candidate_fingerprint: <fingerprint>
```

The candidate exists before controlled execution.

3.5 Policy Snapshot

A governance decision is associated with a specific policy snapshot.

The implementation treats policy identity/version as part of the replay lineage.

If the policy snapshot changes, replay verification against the old lineage fails.

This prevents a decision made under one policy version from being silently represented as having been made under another.

3.6 Governance Decision and Reason Code

The reference implementation supports four decision classes:

Decision	Semantics
ALLOW	The candidate is eligible to be handed to the controlled executor
DENY	The candidate must not proceed
DEFER	Required evidence or conditions are currently insufficient
ESCALATE	Higher authority or human review is required

Every decision is accompanied by a reason code.

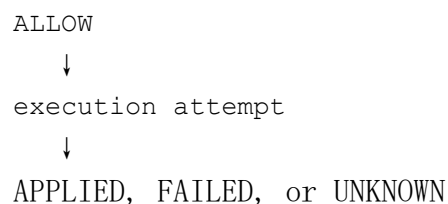
The decision does not represent an execution result.

3.7 Controlled Execution

Only an allowed candidate proceeds to the controlled executor.

In this reference implementation, the executor is purely in memory.

It is used to demonstrate the semantic distinction:



It is not presented as a sandbox, kernel security boundary, production actuator, or hardened enforcement system.

3.8 Observation

Observation reports the result of the controlled execution attempt.

The relevant states are:

```
APPLIED
FAILED
UNKNOWN
NOT_RUN
```

The model obeys the state-advancement rule:

```
if Observation == APPLIED:
    simulated state advances
else:
    simulated state does not advance
```

Therefore:

```
ALLOW → FAILED
```

is a valid outcome.

ALLOW means the attempt was admitted.

It does not mean the attempt succeeded.

3.9 Replayable Lineage Record

Every Lineage Record contains at least:

```
Candidate Transition + fingerprint
State Snapshot + fingerprint
Evidence + fingerprint
Policy Snapshot + fingerprint
Governance Decision
Reason Code
Controlled Execution boundary information
Observation
State After
Record Fingerprint
```

The Replay Verifier checks the original Candidate, State, Evidence, Policy, and Record Fingerprint.

Replay does not invoke the controlled executor again.

Thus:

```
Replay history
≠
Repeat side effect
```

Provenance standards such as PROV provide general structures for representing entities, activities, agents, derivations, and provenance relationships. The present model shares the goal of inspectable lineage but attaches that lineage to an ex-ante candidate-admission decision and a subsequent observation. PROV itself is a general provenance model rather than a candidate-transition admission mechanism. [6]

4. Governance Invariants

The reference model is organized around a small set of explicit semantic invariants.

4.1 Candidate Before Execution

```
No admitted candidate
→ no controlled execution
```

4.2 ALLOW Is Not APPLIED

```
ALLOW ≠ APPLIED
```

An admission decision and an observed execution outcome are different records.

4.3 Observation Controls Simulated State

Advancement

```
APPLIED → advance simulated state
FAILED → do not advance
UNKNOWN → do not advance
NOT_RUN → do not advance
```

4.4 Non-Allow Decisions Do Not Execute

```
DENY      → NOT_RUN
DEFER     → NOT_RUN
ESCALATE  → NOT_RUN
```

4.5 Replay Does Not Re-Execute

```
Replay
→ verify bound lineage
→ no second execution attempt
```

4.6 Binding Mismatches Fail Verification

A mismatch in required governance identity invalidates the expected lineage.

Examples tested by the reference implementation include:

```
wrong evidence object
wrong evidence version
policy snapshot drift
lineage record tampering
```

5. Reference Implementation

5.1 Implementation Scope

The reference implementation is intentionally small.

It is independent of the larger protected engineering system that motivated this line of research.

It does not import:

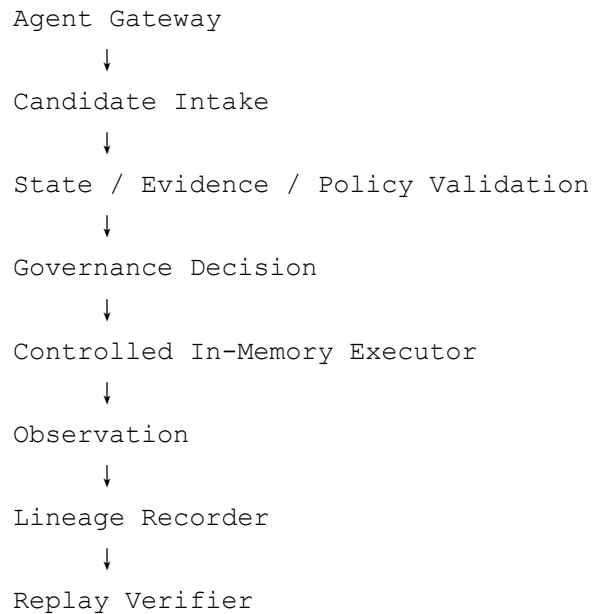
```
protected system source code
protected method libraries
historical internal logs
production keys
provider configuration
deployment topology
```

Its purpose is not to reproduce the protected system.

Its purpose is to reproduce the paper's narrow governance semantics.

5.2 Logical Components

The reference implementation contains the following logical path:



These are logical responsibilities and should not be interpreted as claims about production service decomposition.

5.3 Agent Gateway

The gateway receives Candidate Transitions.

The agent-facing interface does not expose the executor.

A request that attempts to directly call a tool through this interface is denied and audited.

5.4 Controlled Executor

The executor is deterministic and memory-only.

The implementation explicitly maintains:

```
external_side_effects = false
```

The executor can simulate:

```
successful application
execution failure
no execution
```

This is sufficient for testing the separation between governance admission and observation.

5.5 Fingerprint Binding

Fingerprints are used to bind:

```
Candidate Transition
State Snapshot
Evidence
Policy Snapshot
Lineage Record
```

The paper does not evaluate cryptographic attack resistance or claim that the fingerprint implementation forms a production trust root.

It evaluates only the integrity semantics exercised by the fixed tests.

5.6 Protected Engineering Context and Capability Boundary

The public-facing reference model was motivated by governance problems encountered in a larger protected engineering environment involving runtime evidence, derived state, candidate handling, observation, and continuity-oriented review.

That protected environment is not reproduced, benchmarked, or technically disclosed in this release. It is not required to execute, inspect, or validate the claims made by this paper.

Accordingly, the experimental claims in this note are limited to the reference demonstration and its declared protocol.

Figure 2. Protected Engineering Capability Context — Not
Reproduced or Evaluated in This Release

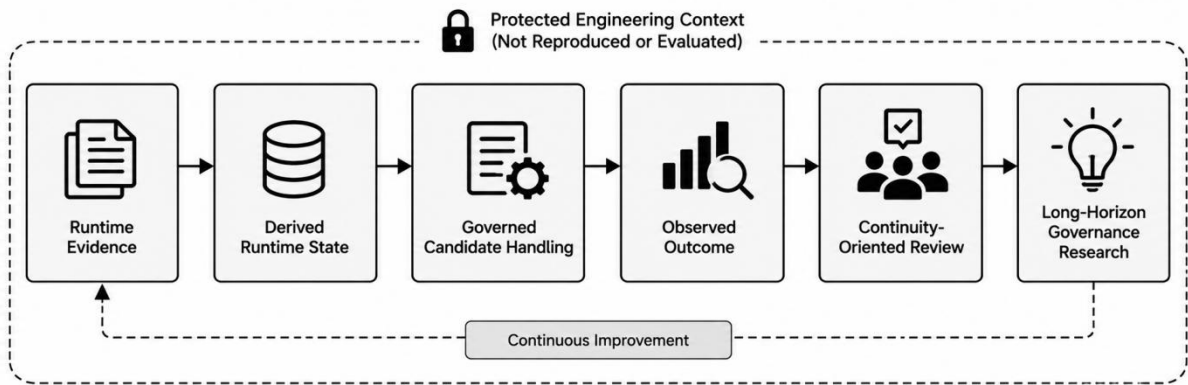


Figure 2 caption. This map is a high-level protected capability context. It does not disclose implementation architecture, algorithms, object schemas, relation computation, source code, internal state machines, method libraries, or operational records. It is not experimental evidence for this note.

5.7 Release Status

The implementation has been tested under:

Python 3.9 → 14 tests passed
 Python 3.13 → 14 tests passed

At the time of this draft, the implementation remains local.

The reference implementation is prepared for controlled public release.

This note does not claim that a public GitHub repository is currently available.

6. Experimental Protocol and Semantic Ablation Design

6.1 Experimental Objective

The purpose of the experiment is not to estimate an empirical safety rate.

The question is:

Does the deterministic reference implementation conform to the governance semantics declared before execution?

The experimental protocol therefore uses fixed, predeclared cases.

6.2 Fixed Cases

Table 1. Predeclared Fixed Cases

ID	Input variation	Expected decision	Expected observation	Simulated effect
C1	Valid evidence, state version, policy, and authorized subject	ALLOW	APPLIED	Yes
C2	Subject not authorized by policy	DENY	NOT_RUN	No
C3	Required evidence missing	DEFER	NOT_RUN	No
C4	Higher-risk candidate requiring higher authority	ESCALATE	NOT_RUN	No
C5	Candidate allowed, simulated execution fails	ALLOW	FAILED	No state advancement
C6	Replay of same bound lineage	Verification succeeds	No new execution	No second effect
I1	Agent attempts direct tool request through public API	DENY	NOT_RUN	No

C1 establishes the admitted success path.

C2 tests subject authorization.

C3 distinguishes insufficient evidence from denial.

C4 tests escalation without execution.

C5 tests the critical distinction:

ALLOW $\neq$ APPLIED

C6 tests replay without re-execution.

I1 tests the public API boundary.

6.3 Semantic Ablation Baseline

To distinguish the contribution of individual lineage bindings from the overall control flow, this note executes a deliberately reduced semantic baseline.

The executed reduced baseline removes:

```
evidence binding
state-version binding
observation-aware state advancement
replay-binding constraints
```

The purpose is not to compare against OPA, OpenShell, or another external product.

It asks a narrower question: if a semantic binding is removed from this same reference model, which declared property is no longer checked?

If a semantic binding is removed from this same reference model, which declared property can no longer be checked?

The four deterministic ablation cases were executed under Python 3.9 and Python 3.13. They show that omitting evidence-context binding accepts wrong-object or wrong-version evidence; omitting current-state-version binding accepts a stale candidate; omitting observation-aware advancement advances the simulated state after FAILED; and omitting bound-lineage replay leaves historical replay not checkable. These are within-model semantic-ablation results, not comparative safety results.

6.4 Integrity and Negative Tests

Table 2. Integrity and Negative Verification

Test	Modification	Expected result	Current status
Evidence Binding Mismatch	Evidence object or version does not match candidate context	DENY	Demonstrated
Policy Snapshot Replay	Snapshot uses changed	Replay	Demonstrated

Test	Modification	Expected result	Current status
Drift	policy snapshot/version	Verification fails	
Record Tampering	Stored Lineage Record modified	Record Fingerprint validation fails	Demonstrated
Direct Tool Bypass	Agent requests direct tool invocation through public API	DENY → NOT_RUN	Demonstrated
Replay Without Re-execution	Valid record is replayed	Verify lineage; do not call executor again	Demonstrated
Idempotency Bound Input Conflict	Candidate, evidence, policy, or DENY; no second effect original state differs under the same idempotency key		Demonstrated

7. Property Verification Results

7.1 Fixed-Case Results

The runner produced:

```
expected decision agreement    = 7/7
expected observation agreement = 7/7
replay verification           = 1/1
```

The same reference implementation passed:

```
Python 3.9 → 14/14 tests
Python 3.13 → 14/14 tests
```

The reference implementation matched all predeclared fixed cases.

The result demonstrates conformance of this deterministic reference model to its declared protocol.

These values are protocol-conformance counts, not estimated real-world safety percentages.

7.2 Governance Property Matrix

Table 3. Property Verification Matrix

Governance property	Reference implementation	Reduced ablation baseline	Supported conclusion
Candidate precedes controlled execution	PASS	NOT APPLICABLE: this binding was retained	Candidate/execution separation is demonstrated in the reference path
Wrong-object or wrong-version evidence does not support transition	PASS	FAIL: wrong evidence accepted without object/version check	Evidence-context binding has observable control semantics in the reference model
Policy drift invalidates historical replay	PASS	NOT CHECKABLE: no bound policy snapshot in the reference lineage replay model verifier	Replay identity is bound to the no bound policy snapshot in the reference lineage replay model
Modified lineage record fails fingerprint validation	PASS	NOT CHECKABLE: no bound checked in the reference model lineage record	Stored lineage integrity is no bound checked in the reference model lineage record
ALLOW → FAILED does not advance state	PASS	FAIL: state advanced without checking	Admission is not conflated with successful application
Replay does not trigger a second execution	PASS	NOT CHECKABLE: replay semantics absent	Verification of history is separated from re-execution
Public API direct-tool request is rejected	PASS	NOT APPLICABLE: API boundary was retained	Agent-facing API does not expose the controlled execution path
Stale candidate state	PASS	FAIL: stale State-version binding	has

Governance property	Reference implementation	Reduced ablation baseline	Supported conclusion
version is checked before execution		candidate accepted without current-version check	observable control semantics in the reference model

The executed ablation establishes only this within-model result: each removed binding corresponds to a governance property that the reduced baseline no longer checks. It does not establish a comparative security result for any external product.

7.3 ALLOW Does Not Mean Success

C5 is a central result.

The candidate receives:

```
Decision = ALLOW
```

but the controlled executor returns:

```
Observation = FAILED
```

The simulated state does not advance.

Therefore:

```
ALLOW ≠ APPLIED
```

is not merely documentation; it is represented in the tested behavior.

7.4 DEFER Is Not DENY

C3 returns:

```
DEFER → NOT_RUN
```

The reference model therefore represents evidence insufficiency separately from a definitive negative authorization result.

This distinction matters because a candidate may be unresolved without being permanently prohibited.

7.5 ESCALATE Is Not Implicit Approval

C4 returns:

```
ESCALATE → NOT_RUN
```

Escalation is therefore represented as a governance outcome requiring a different authority path, not as an implicit success.

7.6 Replay Integrity

The replay verifier checks the identity of:

```
Candidate  
State  
Evidence  
Policy  
Lineage Record
```

A changed Policy Snapshot causes replay verification to fail.

Tampering with the Lineage Record causes Record Fingerprint verification to fail.

Replay itself does not invoke the controlled executor.

7.7 Interpretation Boundary

The fixed results demonstrate only what the declared protocol tests.

They do not demonstrate:

```
real external-tool security  
generalization to unseen tasks  
robustness against hostile same-process code  
kernel-level bypass resistance  
production performance  
independent security certification  
real business-state safety
```

8. Related Work

State-transition governance overlaps with several established research and engineering areas, but this note does not claim to replace them.

8.1 Authorization and Capability Security

Classical protection work established the importance of protected objects, controlled authority, mediation, least privilege, and capability-oriented access to resources. Dennis and Van Horn described protected and shared computational objects in multiprogrammed systems, while Saltzer and Schroeder articulated foundational information-protection principles such as least privilege and complete mediation. [1,2]

NIST's ABAC model determines authorization by evaluating subject, object, operation, and environmental attributes against policy. This means that using contextual state in an authorization decision is not, by itself, novel. The narrower distinction in this note is that a proposed state transition, its pre-state version, supporting evidence, later observation, and replay lineage are bound as one governance object. [5]

8.2 Runtime Policy Enforcement

Schneider's work on enforceable security policies formalized the class of policies enforceable through execution monitoring and security automata. [3]

Open Policy Agent is a general-purpose policy engine that receives structured input, evaluates it against policy and data, and deliberately separates policy decision-making from application-side enforcement. The present note therefore does not argue that policy engines are unable to inspect state or context. Its narrow distinction is the explicit lifecycle of the Candidate Transition through evidence binding, observation, and replay lineage. [7]

8.3 State-Machine and Workflow Modeling

State-transition systems and state machines are long-established tools for modeling reactive systems. Harel's Statecharts introduced hierarchy, concurrency, and communication as extensions to conventional state-transition diagrams. [4]

The contribution of the current note is therefore not the idea of representing state transitions. It is the governance binding around an agent-induced candidate

transition: evidence, pre-state identity, policy snapshot, admission result, controlled execution, observation, and replay lineage.

8.4 Provenance and Auditability

W3C PROV-DM is a general conceptual model for describing provenance of entities, activities, agents, derivations, and responsibility. [6]

The present model uses provenance-like ideas but connects them specifically to an ex-ante governance decision and a later observed execution outcome.

8.5 Agent Runtime Enforcement and Privilege Control

AgentSpec defines a domain-specific language for triggers, predicates, and runtime enforcement mechanisms for LLM agents and was published in the ICSE 2026 Research Track. [8]

Progent focuses on programmable least-privilege policies over LLM-agent tool calls and deterministic privilege constraints. [9]

CaMeL separates trusted control flow from untrusted data flow and applies capabilities to prevent unauthorized information flows and prompt-injection-induced tool misuse. [10]

These works demonstrate that structured runtime control of agent behavior is an active research area. No experimental superiority over these systems is claimed here.

8.6 Sandboxing and External Enforcement

NVIDIA OpenShell is an execution-security system for autonomous agents. Its documented architecture separates the sandboxed agent workload from trusted supervisory components and applies policy to files, system calls, network connections, and credentials. [11]

The present reference implementation does not reproduce such OS- or kernel-level guarantees.

The conceptual relationship is therefore:

Execution / Resource Enforcement

rather than substitution.

8.7 Precise Difference Summary

Table 4. Relationship to Adjacent Approaches

Category	Representative work and scope	Narrow distinction studied here
ABAC	NIST SP 800-162	
	Scope: Authorization using subject/object/action/environment attributes Not claimed to replace: Enterprise access control	Candidate transition is bound to pre-state, evidence, observation, and lineage
Policy Engine	OPA	
	Scope: General structured policy decision Not claimed to replace: Policy languages or PDP/PEP architecture	State, evidence, decision, observation, and replay form one governance lineage
Provenance	W3C PROV	
	Scope: Origin, derivation, responsibility, history Not claimed to replace: General provenance post-execution observation models	Provenance-like records are tied to ex-ante admission and
State Machines	Statecharts	
	Scope: State-transition specification and reactive behavior Not claimed to replace: State-machine transition formalisms	Governance semantics around an agent-induced candidate
Agent Runtime Enforcement	AgentSpec, Progent, CaMeL	
	Scope: Runtime constraints, tool privileges, ALLOW ≠ APPLIED, evidence/state injection defense Not claimed to replace: Agent security lineage replay frameworks	binding, and non-reexecuting
Sandbox Runtime Isolation	OpenShell	
	/ Scope: File/process/network/credential confinement Not claimed to replace: Kernel, sandbox, or change credential isolation	Semantic eligibility and traceability of a candidate state

9. Limitations and Responsible Disclosure

9.1 Fixed Deterministic Case Set

The evaluation contains a small fixed case set.

The result:

7/7

does not mean:

100% real-world safety
universal correctness
generalization
adversarial robustness

It means only that the implementation matched all seven predeclared cases.

9.2 Simulated Executor

The controlled executor is purely in memory.

No conclusion can therefore be drawn about safe governance of:

real APIs
filesystems
shell commands
databases
networks
physical actuators
production workflows

9.3 API Boundary Is Not Process Isolation

The Agent Gateway rejects direct-tool requests made through the public API.

This does not prove protection from malicious same-process code with equivalent Python runtime authority.

9.4 No Hidden-Reasoning Claim

No LLM Chain-of-Thought is read, reconstructed, interpreted, or evaluated.

The term *white-box* refers only to the governance lineage of the external state transition.

9.5 No Comparative Product Benchmark

This release does not experimentally compare against:

OPA
NVIDIA OpenShell
AgentSpec
Progent
CaMeL
commercial policy engines

No superiority claim is made.

9.6 No Method-Evolution Chain

The following chain is out of scope:

Episode
→ Pattern
→ Skeleton
→ Method
→ Operator

9.7 No Unsupported Metadata

This note does not rely on fields such as:

unvalidated confidence scores
M17 or other undeployed method IDs
unpublished philosophical-rule mappings
unvalidated method promotion states

9.8 No Production or Certification Claim

The implementation is not claimed to be:

production-ready
independently security-certified
externally validated
kernel-enforced
hardware-isolated
a complete agent safety system

9.9 Evidence Trust Boundary

Evidence fingerprints and version binding establish consistency within the reference model.

They do not prove that the evidence source itself is honest or uncompromised.

9.10 Protected-System Boundary

The protected engineering context is not a substitute for public reproducibility, external validation, or comparative evidence.

It is presented only to explain the engineering motivation and scope of protected follow-on work.

The evidence categories in this note must therefore remain distinct:

DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION

→ may support Results and Conclusion.

INTERNALLY_IMPLEMENTED_NOT_PUBLICLY_REPRODUCED

→ may describe engineering context only.

RESEARCH_DIRECTION

→ may appear only as future work or discussion.

A capability must not be promoted from one category to another without new evidence.

9.11 Public-Release Boundary

The reference implementation is prepared for controlled public release but is not described here as already available on GitHub.

Before publication, the release should freeze the exact:

source version
test receipts
manifest
file checksums
license
release tag
public disclosure boundary

10. Conclusion

This work establishes a minimal reference model prepared for controlled public release.

It demonstrates specified governance properties only within a deterministic, side-effect-free reference environment.

A larger protected engineering context motivates future integration, continuity, and long-horizon governance research, but those capabilities are not claimed, reproduced, or evaluated here.

The central semantic distinction is:

Proposal
≠ Admission
≠ Execution
≠ Observed Success

and therefore:

Candidate ≠ Commit
ALLOW ≠ APPLIED

The purpose of the model is not to solve agent safety in general. It provides a small, inspectable governance unit for asking a more precise question:

Under this state, this evidence, and this policy snapshot,
was this proposed transition eligible to proceed, what
happened after admission, and can that lineage later be
verified without repeating the effect?

The reference implementation is prepared for controlled public release.

Collaboration inquiries concerning protected engineering capabilities may be directed to the corresponding author.

References

- [1] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966. <https://doi.org/10.1145/365230.365252>.
- [2] J. H. Saltzer and M. D. Schroeder, “The Protection of Information in Computer Systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975. <https://doi.org/10.1109/PROC.1975.9939>.
- [3] F. B. Schneider, “Enforceable Security Policies,” *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000. <https://doi.org/10.1145/353323.353382>.
- [4] D. Harel, “Statecharts: A Visual Formalism for Complex Systems,” *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987. [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9).
- [5] V. C. Hu et al., *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, NIST SP 800-162, 2014, updated 2019. <https://doi.org/10.6028/NIST.SP.800-162>.
- [6] L. Moreau and P. Missier, eds., *PROV-DM: The PROV Data Model*, W3C Recommendation, 30 April 2013. <https://www.w3.org/TR/prov-dm/>.
- [7] Open Policy Agent Project, *Open Policy Agent Documentation*, accessed October 2026. <https://www.openpolicyagent.org/docs/latest/>.
- [8] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents,” in *ICSE ’26*, 2026. <https://doi.org/10.1145/3744916.3764546>.
- [9] T. Shi et al., “Progent: Securing AI Agents with Privilege Control,” *arXiv:2504.11703*, 2025. <https://arxiv.org/abs/2504.11703>.
- [10] E. Debenedetti et al., “Defeating Prompt Injections by Design,” *arXiv:2503.18813*, 2025. <https://doi.org/10.48550/arXiv.2503.18813>.
- [11] NVIDIA, *OpenShell Architecture*, accessed October 2026. <https://docs.nvidia.com/openshell/about/architecture>.

Appendix A. Lineage Record Schema

```
lineage_record:

  candidate_transition:
    value: <candidate transition>
    fingerprint: <candidate fingerprint>

  state_snapshot:
    value: <pre-state snapshot>
    fingerprint: <state fingerprint>

  evidence:
    value: <bound evidence>
    fingerprint: <evidence fingerprint>

  policy_snapshot:
    value: <policy snapshot>
    fingerprint: <policy fingerprint>

  governance:
    decision: ALLOW | DENY | DEFER | ESCALATE
    reason_code: <reason code>

  controlled_execution:
    agent_has_executor_handle: false
    external_side_effects: false
    attempted: true | false

  observation:
    status: APPLIED | FAILED | UNKNOWN | NOT_RUN

  state_after:
    value: <simulated state>

  record_fingerprint:
    value: <record fingerprint>
```

The schema reflects three tested semantic distinctions:

Agent proposal does not directly invoke execution.

ALLOW does not imply APPLIED.

Replay verification does not repeat execution.

Appendix B. Fixed Cases and Expected Outcomes

ID	Condition	Decision	Observation	State advances?
C1	Valid bound transition	ALLOW	APPLIED	Yes
C2	Unauthorized subject	DENY	NOT_RUN	No
C3	Missing evidence	DEFER	NOT_RUN	No
C4	Higher-risk transition	ESCALATE	NOT_RUN	No
C5	Admitted transition, simulated execution fails	ALLOW	FAILED	No
C6	Same bound lineage replayed	Verification succeeds	No new execution	No
I1	Direct tool request	DENY	NOT_RUN	No

Appendix C. Claims-to-Evidence Matrix

Claim	Supporting evidence	Evidence status
Candidate submissions precede controlled execution	Reference path + C1/C5	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Public Agent Gateway does not expose direct executor use	I1	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Unauthorized subject does not execute	C2	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Missing evidence leads to non-execution	C3	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Higher-risk candidate can escalate without	C4	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION

Claim	Supporting evidence	Evidence status
execution		
ALLOW does not imply state advancement	C5	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Only APPLIED advances simulated state	C1 + C5	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Replay does not cause a second effect	C6	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Wrong evidence object/version is rejected	Negative test	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Policy drift invalidates replay	Negative test	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Record tampering is detected by record-fingerprint check	Negative test	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Results reproduce across Python 3.9 and 3.13	14 tests on each version	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION
Protected system contains broader runtime-governance capabilities	Capability-level engineering context only	INTERNALLY_IMPLEMENTED_NOT_PUBLICLY_REPRODUCED
Long-horizon governance integration	Future research	RESEARCH_DIRECTION
Episode→Pattern→Method→Operator evolution	Not evaluated in this note	RESEARCH_DIRECTION
Production external-side-effect safety	No supporting evidence	Excluded
Kernel-level bypass prevention	No supporting evidence	Excluded
Superiority over OPA/OpenShell/AgentSpec/etc.	No comparative experiment	Excluded
Changed bound input under same idempotency key is rejected	Negative subcases replay test	DEMONSTRATED_IN_REFERENCE_IMPLEMENTATION

Appendix D. Unsupported Claims Excluded

The following statements are intentionally excluded from this paper:

1. “The system proves safe AI agents.”
 2. “The system is production-ready.”
 3. “The system prevents all bypasses.”
 4. “The system governs real external side effects.”
 5. “The system outperforms policy engines.”
 6. “The system outperforms NVIDIA OpenShell.”
 7. “The system provides kernel- or hardware-level enforcement.”
 8. “The system exposes or explains LLM chain-of-thought.”
 9. “7/7 means a 100% real-world safety rate.”
 10. “The results prove generalization.”
 11. “The implementation has been independently security-certified.”
 12. “The protected engineering system has been publicly reproduced.”
 13. “The complete method-evolution chain is demonstrated by this paper.”
 14. “The semantic ablation baseline has already been experimentally validated.”
 15. “ALLOW proves that an external state change succeeded.”
 16. “Replay proves the original external effect by executing it again.”
-

Appendix E. Release Metadata

Item	Release status
Author	万根
Affiliation	Independent Researcher
Release version	0.2.2
Citation metadata	CITATION.cff
Code and paper license	LICENSE.md
Public contact channel	593315006@qq.com
Public GitHub repository	Published separately with this release
Git tag	v0.2.2-reference-demo after contact-update freeze
Zenodo record and DOI	Not yet created

Item	Release status
Release manifest and checksums	Generated in this release package
Protected-capability overview	Included without protected implementation detail
Patent and public-disclosure boundary	Requires independent legal review before publication
